Устный доклад к защите TeachAI

Слайды: `Презентация TeachAI.pptx` (20 слайдов).
Заочную колоду на защите не листаем.

Ориентир речи: **около 14 минут**. По методичке закладывают 10–17 минут, на встрече дают 15–20. Текст уложен примерно в 14, чтобы остался запас.

Даты на титуле не зачитывать: там плейсхолдеры `[дата]`.

Как читать файл. Под номером слайда — текст, который можно почти дословно произнести вслух. Строка «Время» — ориентир, не секундомер. Блок «Если спросят» в основную речь не входит.

Если время жмут: экраны 11–13 закрыть одной фразой, а слайды 16 и 18 почти не разворачивать. Минуты на слайдах 5, 14 и 15 лучше не трогать.

Слайды	О чём	Время
1	Титул	0:20
2	Постановка	1:10
3	Цель	0:40
4	Задачи	0:50
5	Данные и модель	1:30
6	Архитектура	1:15
7	Генерация	0:50
8	Контроль качества	1:10
9	Сценарий	0:45
10	Запуск	0:35
11–13	Три экрана входа	0:45
14	Методика	1:40
15	Вердикт	2:00
16	Стек	0:25
17	Без дообучения	0:50
18	Сводка качества	0:30
19	Выводы	0:40
20	Вопросы	0:15
	Всего	**≈ 14:10**

Слайд 1. Титул

Время: 20 секунд.

Добрый день. Меня зовут Игорь Пучкин. Я представляю дипломный проект TeachAI — интеллектуальную систему персонализированного обучения на основе языковой модели. Курс «AI/ML - разработчик», Университет искусственного интеллекта, Пермь.

TeachAI — прикладная система в среде Jupyter. Она работает на готовой языковой модели через API и свою нейросеть не обучает.

Слайд 2. Постановка задачи

Время: 1 минута 10 секунд.

Начну с задачи, которую решает проект.

Чтобы собрать учебный курс, нужно продумать план, написать уроки, подобрать примеры и организовать проверку знаний. Если делать это вручную, работа получается долгой. Готовый одинаковый курс при этом не учитывает, в каком темпе человек занимается и какой стиль объяснения ему удобен.

Поэтому нужна не ещё одна статичная методичка, а система, которая ведёт обучение с помощью уже готовой языковой модели: знает профиль ученика, подстраивает материал и даёт интерактивный курс, а не просто текст на экране.

Слайд 3. Цель проекта

Время: 40 секунд.

Цель проекта — разработать обучающую систему TeachAI: систему, которая генерирует персональный учебный материал с помощью предобученной модели через запросы по API, без своего датасета и без дообучения модели.

Обучение проходит в Jupyter: ученик работает через интерактивный интерфейс в ноутбуке, прогресс обучения сохраняется в JSON-файле, а модель выбрана по результатам эксперимента, о котором скажу позже.

Слайд 4. Задачи проекта

Время: 50 секунд.

Для достижения цели необходимо решить несколько связанных задач.

Нужно генерировать учебный материал — план, уроки, примеры, тесты, ответы и практические задания — с учётом профиля ученика: стиля общения, длительности занятий и общего объёма времени обучения. Всё это должно работать через интерактивный интерфейс в Jupyter. Система проверяет и ответы самой модели перед выдачей ученику — формат плана, синтаксис примеров, уместность вопроса по тексту урока — и ответы ученика на тестах и контрольных заданиях. Прогресс обучения сохраняется в JSON-файле между сессиями.

Отдельно стояла задача экспериментально выбрать модель: сравнить ответы нескольких альтернативных моделей на одной и той же учебной задаче, зафиксировать результаты сравнения и обосновать выбранную для системы модель.

Слайд 5. Данные и модель

Время: 1 минута 30 секунд.

Здесь обычно спрашивают: а где обучающая база?

Отдельной базы под обучение своей нейросети в проекте нет, и это согласовано с куратором. Слева на слайде — то, чем система реально пользуется. Есть готовая языковая модель через API; в рабочем варианте это gpt-4o-mini. Параметры запроса задаём мы. Данные конкретной сессии — профиль, план курса и прогресс обучения — хранятся в JSON-файле.

Справа — то, чего в проекте нет, тоже по согласованию: свой датасет, fine-tuning, RAG и парсинг. Знания уже содержатся в готовой модели. Наша работа — в инструкциях к модели, в коде вокруг неё и в обоснованном выборе самой модели.

Слайд 6. Архитектура системы

Время: 1 минута 15 секунд.

Теперь о том, как система устроена внутри.

Работа начинается со стартового ноутбука: одна ячейка запускает движок — главный модуль системы. Движок связывает остальные модули, и у каждого из них своя функция.

На схеме в центре как раз движок. Модуль конфигурации читает из файла окружения ключ API, имя модели и настройки подключения. Модуль состояния хранит профиль, план курса и прогресс обучения в локальном JSON-файле. Модуль генерации принимает учебные запросы и через единый фасад обращается к языковой модели. Модуль оценивания проводит тесты и проверяет ответы ученика. Модуль интерфейса на виджетах Jupyter показывает экраны урока прямо в ноутбуке.

Стрелка вниз ведёт к внешнему API модели: вызов идёт от системы к модели, а ответ возвращается на тот же запрос. Под схемой подписаны четыре слоя — ядро, модель, интерфейс и сервисный слой. Сервисный слой — это вспомогательные функции внутри Jupyter: логи, индикатор загрузки, оформление HTML и постобработка кода. Отдельного веб-сервиса в проекте нет.

Слайд 7. Генерация контента

Время: 50 секунд.

От архитектуры перейду к тому, как именно формируется учебный материал.

Для формирования всего учебного материала курса система не посылает один-единственный запрос. На каждую задачу в процессе обучения есть отдельный вызов и своя инструкция.

Последовательность запросов такая: план, урок, примеры, тест, вопрос ученика, контрольное задание. Модель при этом одна и та же — gpt-4o-mini. От шага к шагу меняется инструкция, а не веса модели. Так модели проще решать одну понятную задачу, а нам — проверять каждый результат отдельно.

Слайд 8. Промпты и контроль качества

Время: 1 минута 10 секунд.

Поскольку система опирается на языковую модель, ответ нельзя просто принять «как есть» — там, где нужна чёткая структура ответа или рабочий код, выполняется программная проверка ответа, но не по смыслу, а по формату.

Как это реализовано.

Модель возвращает план курса в JSON: программа проверяет, что на месте нужные поля — разделы, темы, уроки — и при необходимости достраивает каркас.

Практические примеры проверяются как код: похож ли текст на Python, проходит ли синтаксис; если нет — запрос можно повторить.

Отдельного смыслового контроля текста урока нет: его качество обеспечивается промптом и качеством самой модели.

Контроль вопросов ученика. Когда ученик задаёт вопрос, система сначала проверяет, относится ли вопрос к тексту урока. Если нет — ответ по существу не запрашивается.

Ответы ученика проверяет модуль оценивания: тест — по эталону, контрольное — по результату выполнения кода.

Оценка «точности генерации» в процентах отдельно не производилась. Насколько конкретные модели подходят для TeachAI, сравнивали в эксперименте — по рубрике, цене и скорости; к нему вернусь чуть позже.

Слайд 9. Сценарий работы

Время: 45 секунд.

Если предыдущие слайды смотрели на систему изнутри, то здесь — тот же путь глазами ученика. На схеме пять шагов.

Сначала заполняется профиль: имя, общее время обучения и стиль общения. Затем строится план курса: модель возвращает план, программа проверяет структуру. Урок либо берётся из уже сохранённой копии, либо генерируется заново. Дальше на уроке ученик может запросить примеры, задать вопрос, пройти тест и выполнить практическое задание. Прогресс обучения записывается в JSON-файл.

При повторном входе ученик видит дашборд. Полная загрузка курса запускается по кнопке, а не автоматически при каждом открытии.

Слайд 10. Запуск системы

Время: 35 секунд.

Точка входа в систему — ноутбук TeachAI. Одна стартовая ячейка поднимает интерактивный интерфейс. Для запуска нужны ключ API в файле окружения и ядро Python.

Сам ноутбук на защите я открывать не буду. Дальше покажу три экрана одной реальной учебной сессии.

Слайд 11. Начало работы — профиль

Время: 15 секунд.

Первый экран — профиль. Ученик вводит имя, общее время, длительность одного занятия и стиль общения, затем сохраняет настройки. Цифры 31 час и 30 минут на экране относятся к этой конкретной сессии. Это не значения «по умолчанию» для всех учеников.

Слайд 12. Начало работы — выбор курса

Время: 15 секунд.

На следующем экране курс выбирается из каталога. Рядом показаны описание и требования. Этот текст берётся из каталога курсов, а не сочиняется моделью на лету.

Слайд 13. Начало работы — старт

Время: 15 секунд.

На третьем экране ученик видит сводку настроек и кнопку «Начать курс». После этого система ведёт его уже по составленному плану.

Слайд 14. Сравнение моделей. Методика

Время: 1 минута 40 секунд.

Перейду к выбору модели — к той задаче, о которой говорил в начале.

Ответить на вопрос «какую модель взять» только по личному впечатлению было бы ненадёжно. Поэтому 13 сентября 2026 года я провёл сравнение.

Сравнивали одно и то же: генерацию урока по одной и той же инструкции. Участвовали пять тем курса «Основы Python» и пять моделей: gpt-4o-mini, gpt-5.6-luna, gpt-5.6-terra, Claude Sonnet 5 и Gemini Flash.

Сначала работал фильтр формы: проверялись объём текста, заголовки и наличие блока кода Python. Этот фильтр прошли все 25 уроков из 25. Значит, дальше сравнивалось уже содержание, а не умение аккуратно упаковать ответ.

Содержание оценивали три модели-судьи разных семейств. Они видели ответы под слепыми метками. Это gpt-5.6-sol, claude-opus-5 и gemini-3.1-pro. Оценка шла по рубрике от 0 до 10: тема, полнота, плотность, педагогика и соответствие заданию.

Есть важная оговорка. Судьям текст обрезали примерно до 3500 символов, чтобы условия сравнения были одинаковыми. Цена и скорость при этом считались по полному ответу. Оценивался именно тип «урок», а не весь учебный конвейер системы целиком.

Слайд 15. Результаты и выбор модели

Время: 2 минуты.

Теперь результаты. Таблицу я читаю не как рейтинг «кто умнее вообще», а по критериям, которые были зафиксированы ещё до прогона: балл на рубль, цена, скорость и приемлемое качество.

В системе остаётся gpt-4o-mini. У неё 7,73 балла, 53 копейки за урок, 14,72 балла на рубль и около 12 секунд на ответ. По баллу на рубль и по скорости это лучший результат в пятёрке.

Выше по абсолютному баллу стоит Luna: 8,40, около 88 копеек за урок, 9,57 балла на рубль, но уже около 84 секунд. Разница в качестве небольшая, а по цене и особенно по скорости для учебного проекта выгоднее mini. Luna я оставляю запасным вариантом на случай, если появится бюджет и можно будет подождать дольше.

Остальные три модели по баллу стоят рядом с лидерами, но балл на рубль у них около единицы: у Flash 1,28, у Terra 0,94, у Claude 1,01. Для массовой генерации уроков это невыгодно.

Итоговый вывод такой: в системе оставляем gpt-4o-mini; если понадобится апгрейд при большем бюджете — берём gpt-5.6-luna.

Этой фразы на слайде нет. Её можно сказать, если останется полминуты, или оставить на вопрос: на одном пробном уроке лидировал Claude, а на полном прогоне из пяти уроков лидер по качеству — Luna. Поэтому решение принято именно по полному прогону.

Слайд 16. Технологический стек

Время: 25 секунд.

Коротко о стеке. Я выбирал его под задачу и без лишнего. Система написана на Python и работает в Jupyter с виджетами интерфейса. Языковая модель вызывается через API; в продукте это gpt-4o-mini. Настройки лежат в файле окружения, состояние сессии — в JSON-файле. Сравнение моделей вынесено в отдельный ноутбук, чтобы эксперимент не смешивался с обычной работой ученика.

Слайд 17. Предобученная LLM без дообучения

Время: 50 секунд.

Ещё раз коротко зафиксирую принцип проекта. Его иногда понимают неправильно: будто мы просто не успели обучить свою нейросеть.

На самом деле задача была другой: построить систему обучения, а не тренировать нейросеть. На схеме три звена: готовая модель, промпты и код TeachAI, а на выходе — учебный материал. Своего GPU и размеченного корпуса у проекта нет. Тема курса при этом может быть любой, потому что знания уже есть в модели. Этот подход согласован с куратором.

Слайд 18. Обеспечение качества

Время: 30 секунд.

Здесь короткая сводка. Новых цифр на слайде нет.

В самой системе качество держат программные проверки: структура JSON-плана, синтаксис примеров, фильтр вопросов, кэш и повтор запроса

при ошибке. В эксперименте — рубрика судей, фильтр формы, цена и скорость. Это две разные линии контроля, и обе нужны.

Слайд 19. Выводы

Время: 40 секунд.

Подведу итог.

Собрана модульная система TeachAI в Jupyter. Учебный материал формируется последовательностью отдельных запросов к модели. Есть персонализация, а прогресс обучения сохраняется в JSON-файле. Модель gpt-4o-mini выбрана по результатам эксперимента: у неё лучший балл на рубль и лучшая скорость. Работа на готовой модели без дообучения согласована с куратором, этап 5 принят.

Слайд 20. Заключение

Время: 15 секунд.

Спасибо за внимание. Готов ответить на вопросы.

Планов на будущее на этом слайде нет. Сайт, смена модели и расширение эксперимента в сданный проект не входят, и я их здесь не обещаю.

Если спросят

Этот блок в основную речь не входит. Ответы тоже связные, но короче.

Почему не обучали свою нейросеть.
Задача проекта — вести ученика по курсу, а не подбирать веса нейросети. Свой датасет, fine-tuning, RAG и парсинг согласованно не применяются. Этапы сбора базы и парсинга закрыты этим объяснением. Этап 5 с финальной системой и экспериментом принят.

Почему mini, если у Luna балл выше.
Для учебного проекта важны балл на рубль и скорость при приемлемом качестве. 7,73 из 10 — это не провал. Luna лучше по абсолютному баллу и близка по цене, но отвечает около 84 секунд против 12 у mini. Её имеет смысл брать, если приоритет сместится к качеству и можно подождать.

Почему нельзя было решить по одному уроку.
На пробном уроке лидировал Claude, около 9 баллов. На полном прогоне из пяти уроков лидер по качеству — Luna, 8,40. Поэтому вердикт снят именно с полного прогона.

Что значит обрезка до 3500 символов.
Судьям отдавали начало урока, чтобы условия сравнения были одинаковыми. Хвост длинного текста в оценку входил слабее. Цена и скорость считались по полному ответу. Сравнивали генерацию урока, а не план, не тест и не весь интерфейс.

Из чего складывается балл 0–10.
Пять критериев по шкале от 0 до 2: соответствие теме, полнота, плотность, педагогика и соответствие заданию. Судьи видели слепые метки, а не имена моделей.

Что такое Luna, Terra и имена судей.
Это идентификаторы моделей в прогоне от 13 сентября. В работающей системе стоит gpt-4o-mini.

Что значат проценты, если куратор помнит заочную версию.
Порог зачёта теста — больше 40%. Урок считается закрытым, когда выполнены оба условия: тест зачтён и контрольное задание принято. 80% на экране теста — балл одной попытки. 62,5% на дашборде — доля пройденных уроков этой сессии, то есть 5 из 8. Это не точность модели.

Как устроен вопрос ученика.
Сначала отдельный запрос проверяет, относится ли вопрос к тексту урока. Если нет, показывается предупреждение, и ответ по существу не запрашивается. Если да, идёт второй запрос — уже за самим ответом.

Про RAG «в развитии».
В сданном проекте RAG нет, и в планы защиты я его не ставлю.